

UNIVERSITY OF MILANO-BICOCCA

Department of Informatics, Systems and Communication

Advanced Data Management Project Report

City Mobility Platform — Data Management System

Academic Year 2025/2026

Instructors: Gloria Lopiano & Andrea Campagner

Student Name: Md Rifatul Haque

Matriculation Number: 947512

Submission Date: 6 June 2026

Table of Contents

Part 1: Relational vs Document Model	4
1.1 Data Modeling	4
Relational Schema — PostgreSQL	4
Document Schema — MongoDB	5
Why Embedding vs Referencing?	5
1.2 Query Implementation	5
Query 1 — Show all trips with user details and station names	6
Query 2 — Show each user with their trip count and average trip duration	7
Query 3 — Show each station with how many trips start and end there	9
Query 4 — Find all trips that have at least one ERROR event	10
Benchmark Results — All 36 Scale Combinations	12
What the Results Tell Us	14
1.3 Schema Evolution — Adding a New Field	14
PostgreSQL — How to add the field	14
MongoDB — How to add the field	15
1.4 Spark Implementation of Query 2	17
Part 2: Graph Model	18
2.1 Graph Schema and Queries	18
Graph Design	18
Cypher Query 1 — All stations reachable by a given user	20
Cypher Query 2 — Top 3 most important stations (by trip count)	20
Neo4j Scalability Benchmark — All 36 Combinations	21
2.2 Spark Graph Queries	22
PageRank — Finding the Most Important Stations	22
Connected Components — Is the Network Connected?	24
Spark Graph Scalability Benchmark — All 36 Combinations	24
Part 3: Partitioning and Replication	25
3.1 Partitioning	26
Strategy A: Partition by user_id	26
Strategy B: Partition by start_station_id	26
Our Recommendation: Partition by user_id	27
3.2 Replication	27
How to Reduce These Risks	28
Conclusion	28

Part 1: Relational vs Document Model

In this part, we build the same database using two different approaches: a traditional relational database (PostgreSQL) and a document database (MongoDB). We then compare how well each one handles our four queries at different data sizes.

1.1 Data Modeling

Relational Schema — PostgreSQL

In PostgreSQL, data is organized into four separate tables that are linked together using foreign keys (like references between tables):

- users — stores each user's name, surname, birthdate, and country
- stations — stores each station's name, city, and maximum vehicle capacity
- trips — stores each trip, linking a user to a start station and end station, with start/end times and cost
- events — stores each event that happened during a trip (GPS, ERROR, BATTERY, or DELAY)

Indexes were added on all linking columns (foreign keys) and on the event type column to make queries faster.

```
10
11 -- -----
12 -- USERS
13 -- -----
14 CREATE TABLE users (
15     user_id SERIAL PRIMARY KEY,
16     name VARCHAR(100) NOT NULL,
17     surname VARCHAR(100) NOT NULL,
18     birthdate DATE NOT NULL,
19     country VARCHAR(100) NOT NULL
20 );
21
22 -- -----
23 -- STATIONS
24 -- -----
25 CREATE TABLE stations (
26     station_id SERIAL PRIMARY KEY,
27     name VARCHAR(150) NOT NULL,
28     city VARCHAR(100) NOT NULL,
29     capacity INTEGER NOT NULL CHECK (capacity > 0)
30 );
31
32 -- -----
33 -- TRIPS
34 -- -----
35 CREATE TABLE trips (
36     trip_id SERIAL PRIMARY KEY,
37     user_id INTEGER NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
38     start_station INTEGER NOT NULL REFERENCES stations(station_id) ON DELETE RESTRICT,
39     end_station INTEGER NOT NULL REFERENCES stations(station_id) ON DELETE RESTRICT,
40     start_time TIMESTAMP NOT NULL,
41     end_time TIMESTAMP NOT NULL,
42     total_cost NUMERIC(8,2) NOT NULL CHECK (total_cost >= 0),
43     CONSTRAINT chk_trip_times CHECK (end_time > start_time)
44 );
45
46 -- -----
47 -- EVENTS
48 -- -----
49 CREATE TABLE events (
50     event_id SERIAL PRIMARY KEY,
51     trip_id INTEGER NOT NULL REFERENCES trips(trip_id) ON DELETE CASCADE,
52     timestamp TIMESTAMP NOT NULL,
53     event_type VARCHAR(10) NOT NULL CHECK (event_type IN ('GPS', 'ERROR', 'BATTERY', 'DELAY')),
54     value TEXT NOT NULL
55 );
56
```

Figure 1: Relational Schema (PostgreSQL)

Document Schema — MongoDB

In MongoDB, data is stored as documents (similar to JSON files). We use three collections:

- users collection — one document per user
- stations collection — one document per station
- trips collection — one document per trip, with all events stored inside the trip document (embedded)

The most important design decision is that events are stored inside their trip document. This is called embedding. Users and stations are stored separately and referenced by ID. This is called referencing.



```
83 def create_collections():
84     """Create collections with schema validators and indexes."""
85
86     # — users —————
87     try:
88         db.create_collection("users", validator=users_validator)
89         print("Collection 'users' created.")
90     except CollectionInvalid:
91         db.command("collMod", "users", validator=users_validator)
92         print("Collection 'users' already exists - validator updated.")
93
94     db.users.create_index([("surname", ASCENDING), ("name", ASCENDING)])
95     db.users.create_index([("country", ASCENDING)])
96
97     # — stations —————
98     try:
99         db.create_collection("stations", validator=stations_validator)
100        print("Collection 'stations' created.")
101    except CollectionInvalid:
102        db.command("collMod", "stations", validator=stations_validator)
103        print("Collection 'stations' already exists - validator updated.")
104
105    db.stations.create_index([("city", ASCENDING)])
106
107    # — trips (events embedded) —————
108    try:
109        db.create_collection("trips", validator=trips_validator)
110        print("Collection 'trips' created.")
111    """
```

Figure 2: Document Schema (MongoDB)

Why Embedding vs Referencing?

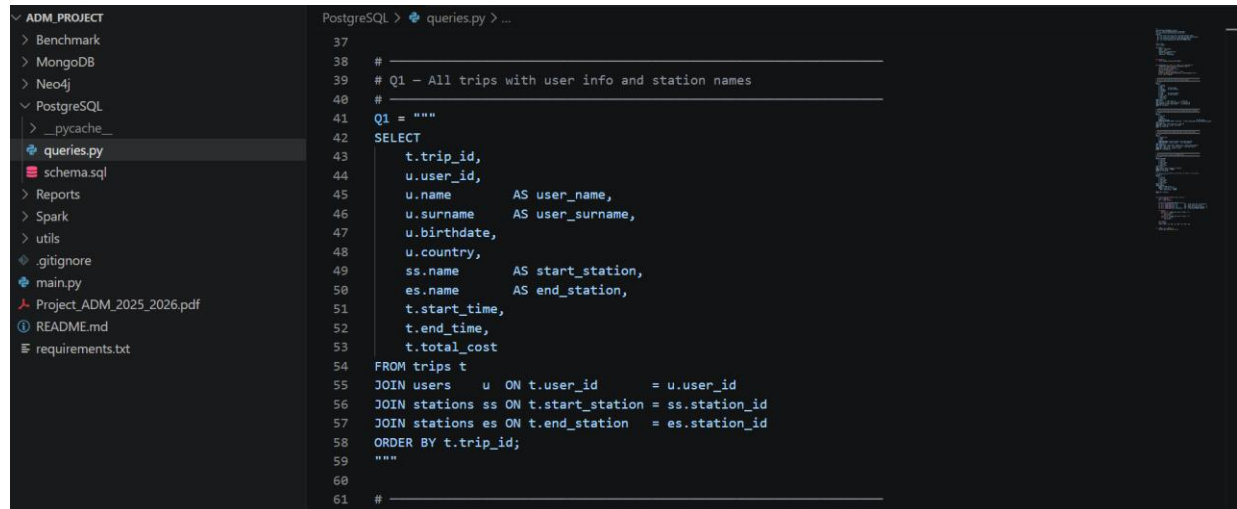
What	Decision	Why
Events	EMBEDDED inside trips	Events only make sense as part of a trip. Storing them inside the trip avoids slow joins.
Users	REFERENCED (separate)	One user can have many trips. If we embedded user info in every trip, we would repeat the same data thousands of times.
Stations	REFERENCED (separate)	Same reason — only ~20 stations serve thousands of trips. Repeating station data would waste space.

1.2 Query Implementation

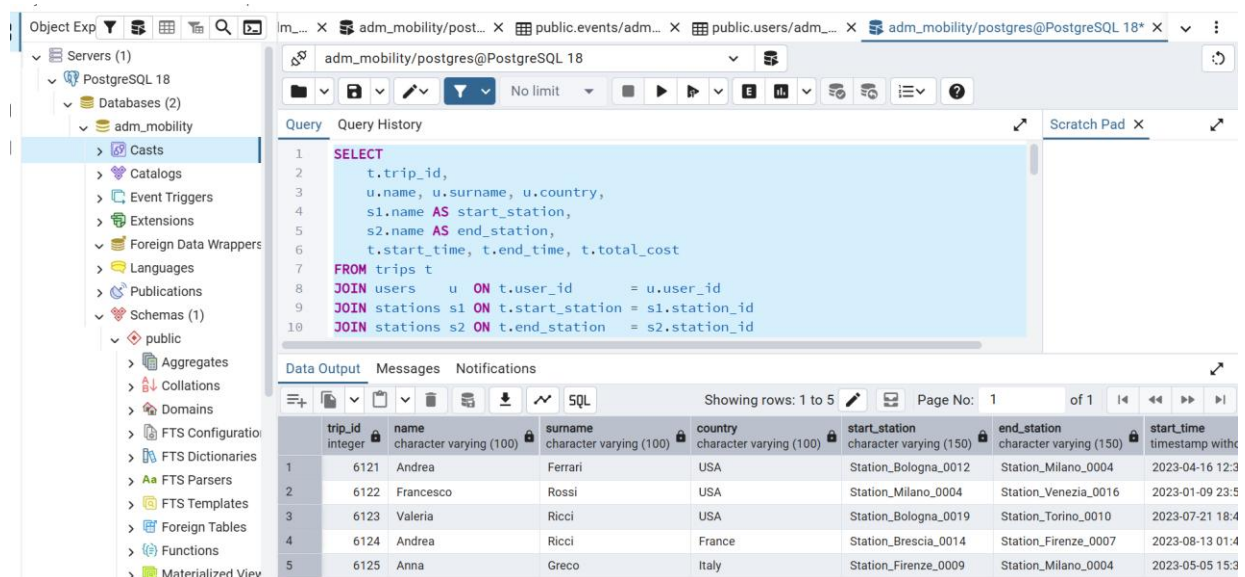
We implemented the same four queries in both PostgreSQL and MongoDB, then measured how long each one takes at different data sizes.

Query 1 — Show all trips with user details and station names

PostgreSQL Approach: We join three tables together — trips, users, and stations (twice for start and end). Because all linking columns are indexed, this is fast.



```
37
38 #
39 # Q1 - All trips with user info and station names
40 #
41 Q1 = """
42 SELECT
43     t.trip_id,
44     u.user_id,
45     u.name      AS user_name,
46     u.surname   AS user_surname,
47     u.birthdate,
48     u.country,
49     ss.name     AS start_station,
50     es.name     AS end_station,
51     t.start_time,
52     t.end_time,
53     t.total_cost
54 FROM trips t
55 JOIN users u ON t.user_id = u.user_id
56 JOIN stations ss ON t.start_station = ss.station_id
57 JOIN stations es ON t.end_station = es.station_id
58 ORDER BY t.trip_id;
59 """
60
61 #
```



trip_id	name	surname	country	start_station	end_station	start_time
6121	Andrea	Ferrari	USA	Station_Bologna_0012	Station_Milano_0004	2023-04-16 12:3
6122	Francesco	Rossi	USA	Station_Milano_0004	Station_Venezia_0016	2023-01-09 23:5
6123	Valeria	Ricci	USA	Station_Bologna_0019	Station_Torino_0010	2023-07-21 18:4
6124	Andrea	Ricci	France	Station_Brescia_0014	Station_Firenze_0007	2023-08-13 01:4
6125	Anna	Greco	Italy	Station_Firenze_0009	Station_Milano_0004	2023-05-05 15:3

Figure 3: PostgreSQL - Q1: All trips with user information and station names

MongoDB Approach: We use three \$lookup operations (similar to joins). This is slower than PostgreSQL because MongoDB has to look up referenced documents separately at query time.

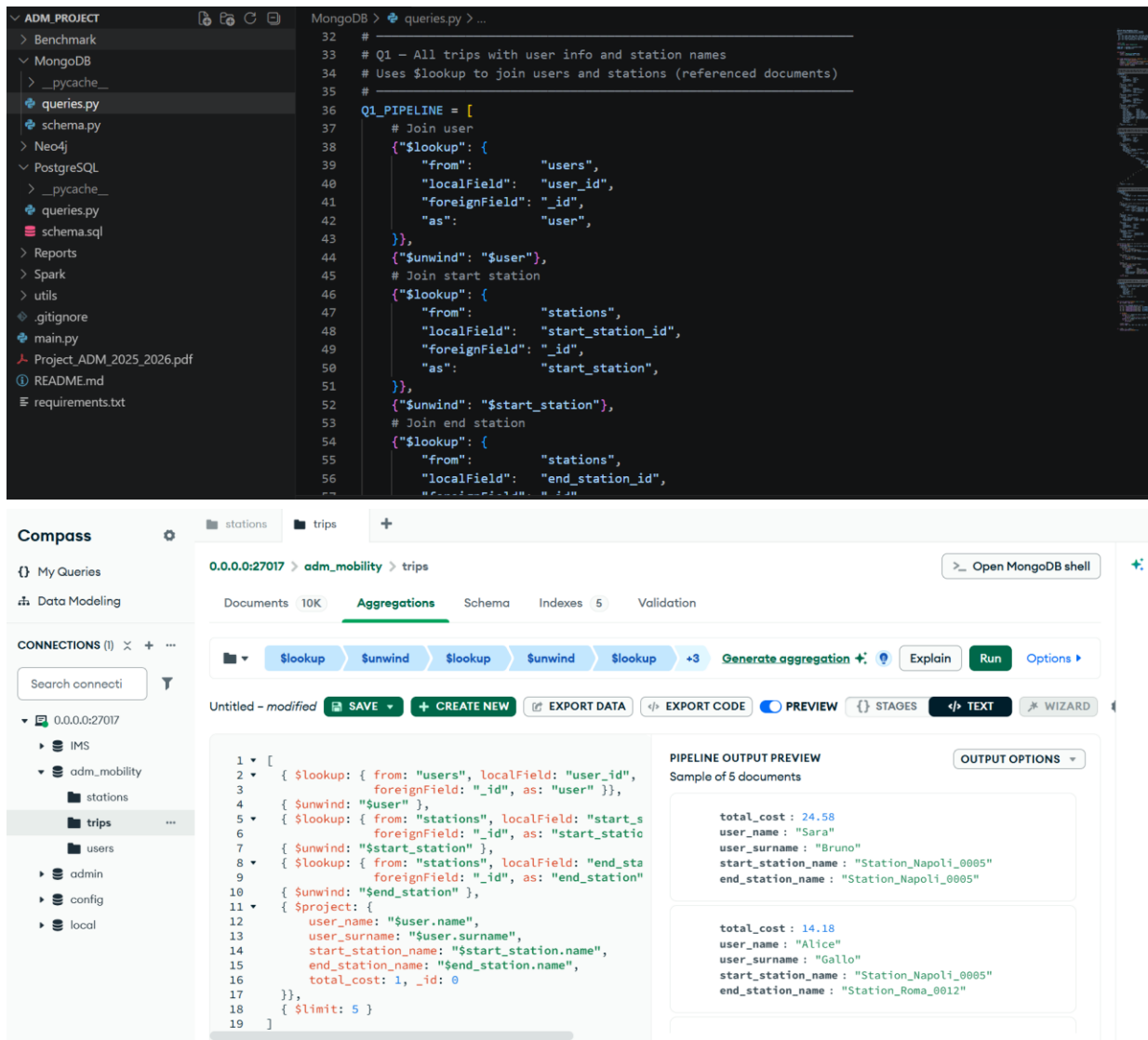
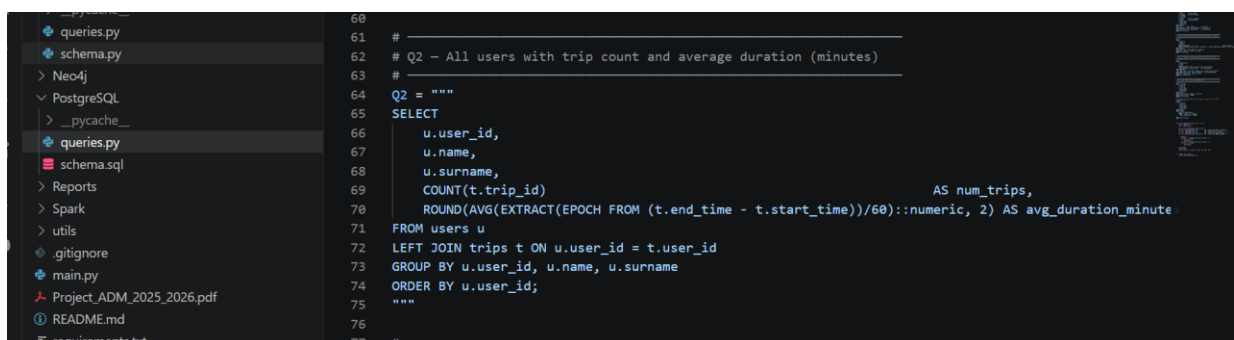


Figure 4: MongoDB - Q1: All trips with user information and station names

Query 2 — Show each user with their trip count and average trip duration

PostgreSQL Approach: Group all trips by user and calculate COUNT and AVG. Very efficient with indexed columns.



The screenshot shows the PostgreSQL DBeaver interface. On the left, the 'Schemas (1)' pane shows the 'public' schema. The main query editor contains the following SQL query:

```

1 SELECT
2     u.user_id, u.name, u.surname,
3     COUNT(t.trip_id) AS num_trips,
4     ROUND(AVG(EXTRACT(EPOCH FROM (t.end_time - t.start_time))/60)::numeric, 2) AS
5 FROM users u
6 LEFT JOIN trips t ON u.user_id = t.user_id
7 GROUP BY u.user_id, u.name, u.surname
8 ORDER BY u.user_id;

```

Below the query editor, the 'Data Output' tab shows the results of the query. The table has 6 columns: user_id, name, surname, num_trips, and avg_duration_minutes. The results are as follows:

user_id	name	surname	num_trips	avg_duration_minutes
1	Giulia	Esposito	6	112.67
2	Anna	Ricci	3	100.67
3	Davide	Rossi	11	78.55
4	Marco	Colombo	9	99.11
5	Marco	Greco	4	70.75

The status bar at the bottom indicates 'Total rows: 1000' and 'Query complete 00:00:00.213'.

Figure 5: PostgreSQL - Q2: Users with number of trips and average trip duration

MongoDB Approach: Look up each user's trips and compute statistics. Slower when there are many users (50k) because of the lookup overhead.

The screenshot shows the MongoDB Atlas interface. The left sidebar displays the project structure, including 'ADM_PROJECT', 'MongoDB', and 'queries.py'. The main editor shows a Python script for a MongoDB query:

```

79 # Q2 - Users with trip count and average duration
80 # Start from users collection and lookup trips
81 #
82 Q2_PIPELINE = [
83     # For each user, lookup their trips
84     {"$lookup": {
85         "from": "trips",
86         "localField": "_id",
87         "foreignField": "user_id",
88         "as": "trips",
89     }},
90     # Compute stats
91     {"$project": {
92         "name": 1,
93         "surname": 1,
94         "num_trips": {"$size": "$trips"},
95         "avg_duration_minutes": {
96             "$cond": [
97                 {"$gt": [{"$size": "$trips"}, 0]},
98                 {
99                     "$divide": [
100                         {"$avg": {
101                             "$map": {
102                                 "input": "$trips",
103                                 "as": "t",
104                                 "in": {
105                                     "$divide": [
106                                         {"$subtract": [{"t.end_time", "t.start_time"}]},
107                                         60
108                                     ]
109                                     },
110                                 "as": "avg_duration_minutes"
111                             }
112                         },
113                         {"$size": "$trips"}
114                     ]
115                 }
116             ]
117         }
118     }},
119 ]

```

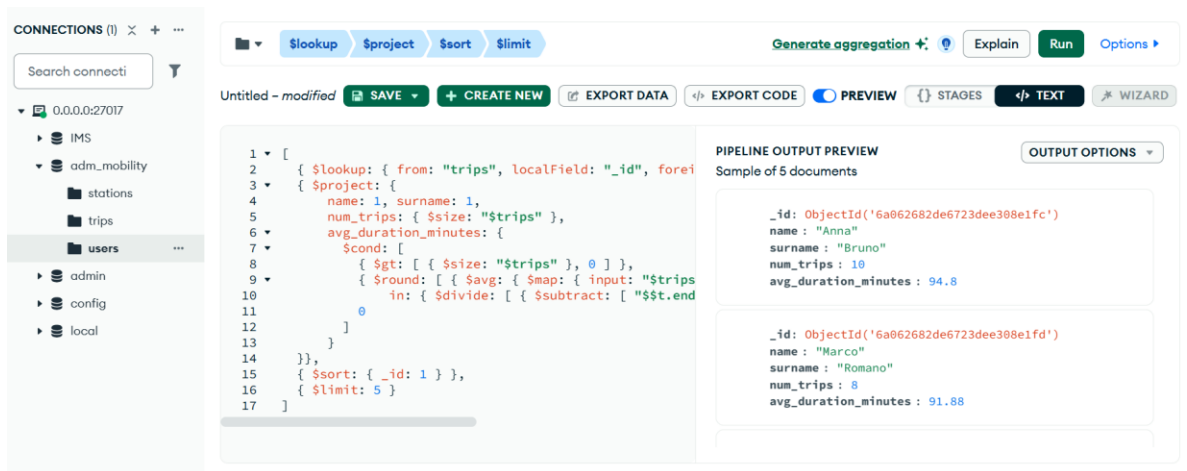



Figure 6: MongoDB - Q2: Users with number of trips and average trip duration

Query 3 — Show each station with how many trips start and end there

PostgreSQL Approach: Join the trips table twice (once for start, once for end). This creates a very large intermediate result, which makes it extremely slow at large scales.

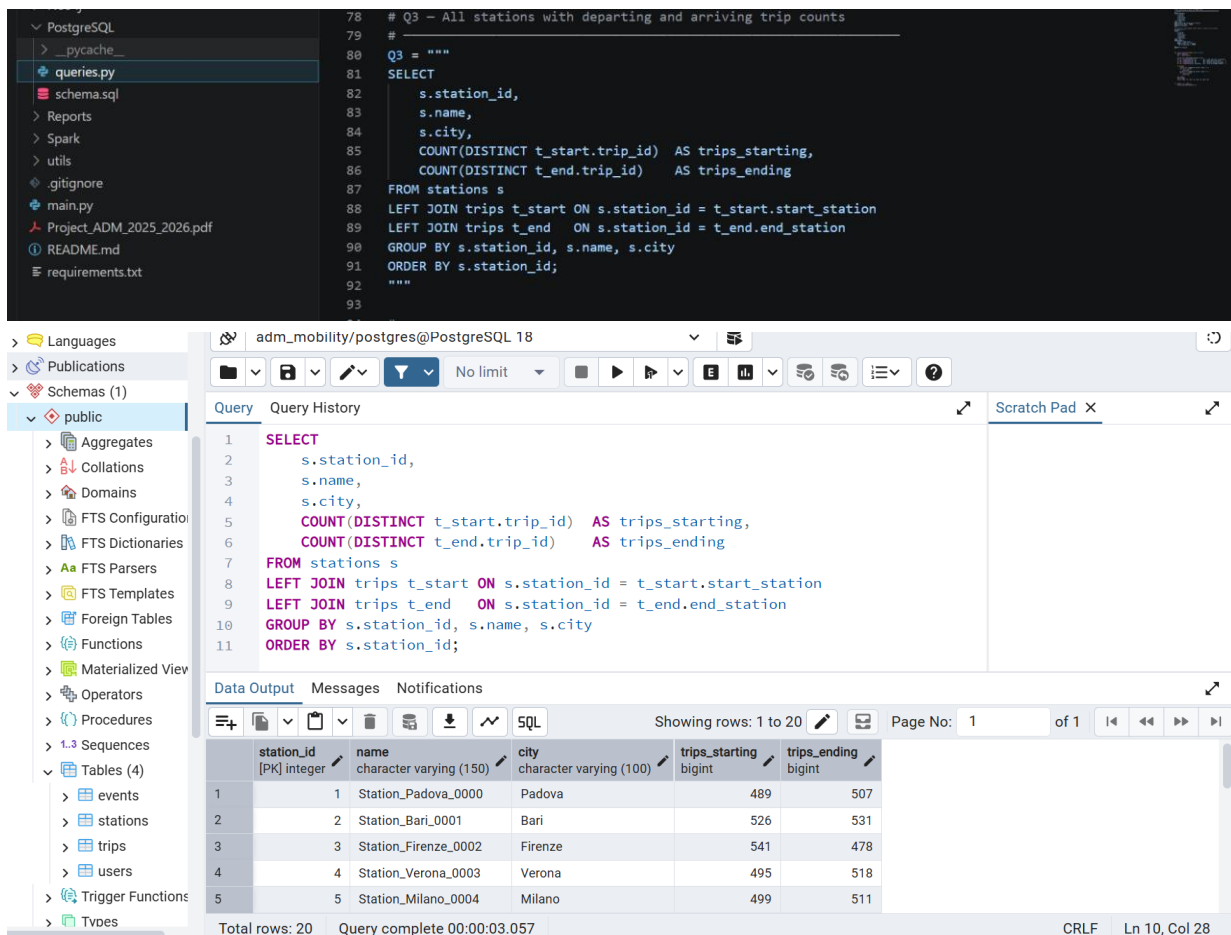


Figure 7: PostgreSQL - Q3: Stations with trips starting and ending there

MongoDB Approach: Use \$facet to count start trips and end trips separately and efficiently. This is much, much faster than PostgreSQL at large scales.

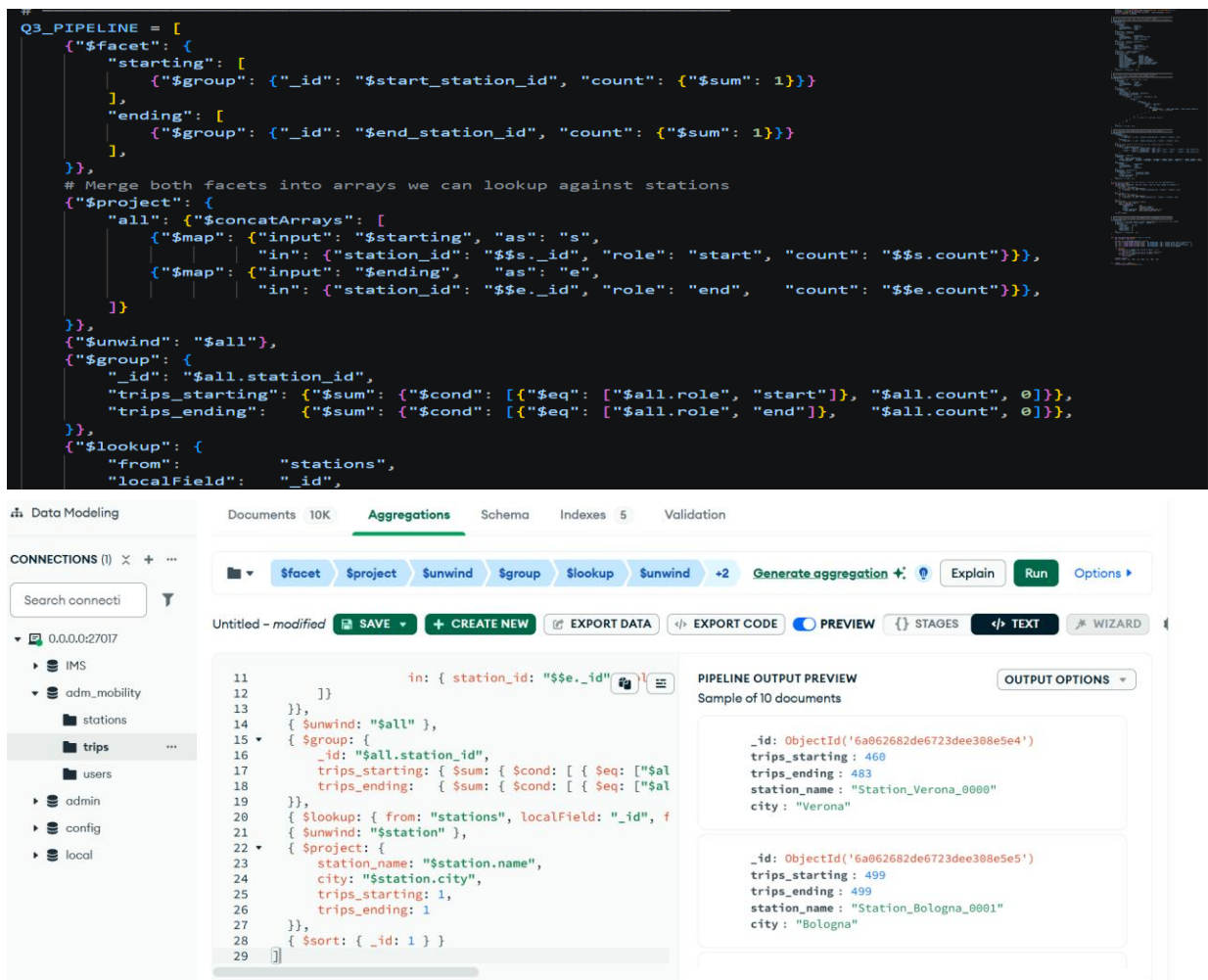
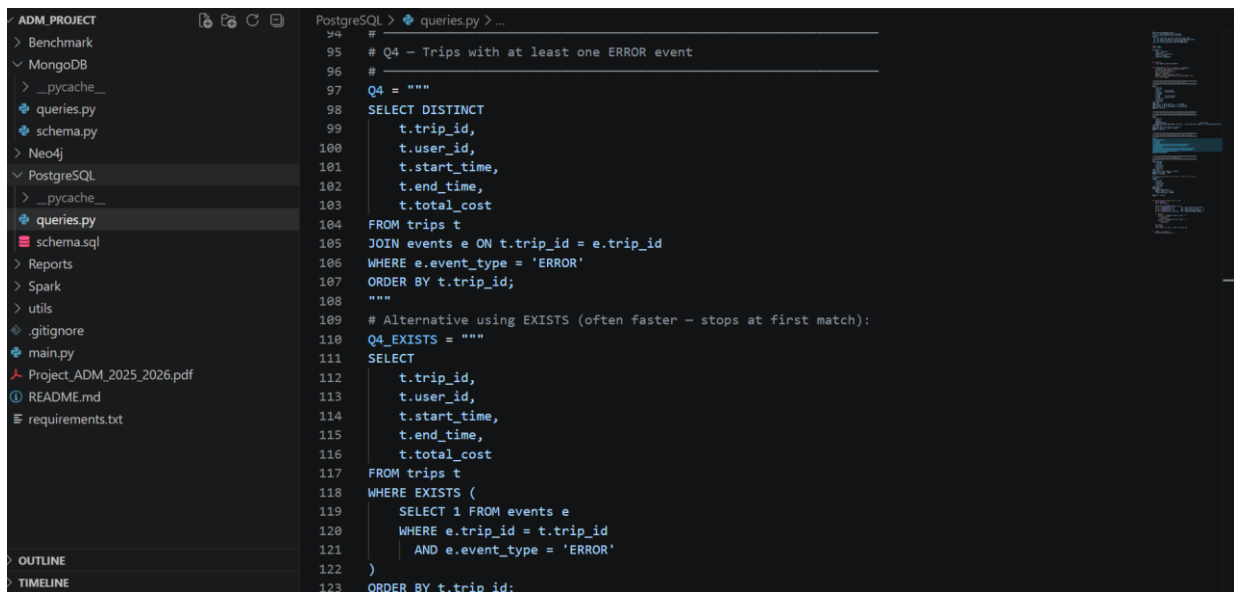


Figure 8: MongoDB -Q3: Stations with trips starting and ending there

Query 4 — Find all trips that have at least one ERROR event

PostgreSQL Approach: Use an EXISTS check on the events table — stops as soon as it finds one matching event, so it is efficient.



Query

```

2  t.trip_id,
3  t.total_cost,      -- moved near the front so it always shows
4  t.user_id,
5  t.start_time,
6  t.end_time
7  FROM trips t
8  WHERE EXISTS (
9      SELECT 1 FROM events e
10     WHERE e.trip_id = t.trip_id
11           AND e.event_type = 'ERROR'
12 )
13 ORDER BY t.trip_id;

```

Data Output

trip_id [PK] integer	total_cost numeric (8,2)	user_id integer	start_time timestamp without time zone	end_time timestamp without time zone
1	1	16.49	914	2023-04-14 10:25:33
2	3	3.23	288	2023-09-01 13:24:37
3	7	7.73	214	2023-02-09 18:35:10
4	14	8.01	822	2023-07-30 12:58:29
5	16	6.62	453	2023-09-18 05:06:08

Total rows: 4352 Query complete 00:00:00.137 CRLF Ln 13, Col 20

Figure 9: PostgreSQL - Q4: Trips containing at least one ERROR event

MongoDB Approach: Filter directly on the embedded events array. Because events are inside the trip document, no separate table lookup is needed.

```

195 #
196 # Q4 - Trips with at least one ERROR event (embedded array filter)
197 #
198 Q4_PIPELINE = [
199     # Filter trips that have at least one embedded event with type ERROR
200     {"$match": {"events.event_type": "ERROR"}},
201     {"$project": {
202         "trip_id": "$_id",
203         "user_id": 1,
204         "start_time": 1,
205         "end_time": 1,
206         "total_cost": 1,
207     }},
208     {"$sort": {"trip_id": 1}},
209 ]
210
211 def run_all_queries(verbose: bool = False):
212     db, client = get_db()
213
214     print("\n=== MongoDB Queries ===\n")
215     r1, t1 = timed_pipeline(db.trips, Q1_PIPELINE, "Q1 - Trips with user & stations")
216     r2, t2 = timed_pipeline(db.users, Q2_PIPELINE, "Q2 - Users with trip stats")
217     r3, t3 = timed_pipeline(db.trips, Q3_PIPELINE, "Q3 - Stations with trip counts")
218     r4, t4 = timed_pipeline(db.trips, Q4_PIPELINE, "Q4 - Trips with ERROR")
219
220     if verbose:
221         print("\n--- Sample Q1 (first 2 docs) ---")
222         for doc in r1[:2]:
223

```

0.0.0.0:27017 > adm_mobility > trips

Documents 10K Aggregations Schema Indexes 5 Validation

Generate aggregation Explain Run Options

Untitled - modified SAVE + CREATE NEW EXPORT DATA EXPORT CODE PREVIEW STAGES TEXT WIZARD

```

1  [
2  {
3    { $match: { "events.event_type": "ERROR" } },
4    { $project: {
5        trip_id: "$_id",
6        total_cost: 1,
7        user_id: 1,
8        start_time: 1,
9        end_time: 1,
10       _id: 0
11     }},
12     { $sort: { trip_id: 1 } },
13     { $limit: 5 }
14 ]

```

PIPELINE OUTPUT PREVIEW

Sample of 5 documents

```

user_id: ObjectId('6a062682de6723dee308e26a')
start_time: 2023-02-25T18:05:06.000+00:00
end_time: 2023-02-25T19:35:06.000+00:00
total_cost: 11.8
trip_id: ObjectId('6a062682de6723dee308e5fc')

user_id: ObjectId('6a062682de6723dee308e2b8')
start_time: 2023-09-10T22:36:45.000+00:00
end_time: 2023-09-11T00:47:45.000+00:00
total_cost: 18.21
trip_id: ObjectId('6a062682de6723dee308e5fd')

```

Figure 10: MongoDB - Q4: Trips containing at least one ERROR event

Benchmark Results — All 36 Scale Combinations

We tested every combination of: 1k/10k/50k users × 10k/50k/100k trips × 0/2/5/10 events per trip. Times shown in seconds (lower = faster):

DB	Users	Trips	Events	Q1 (s)	Q2 (s)	Q3 (s)	Q4 (s)
PG	1k	10k	0	0.20	0.02	3.92	0.02
MG	1k	10k	0	2.02	0.06	0.02	0.01
PG	1k	10k	2	0.50	0.01	3.26	0.16
MG	1k	10k	2	2.19	0.04	0.02	0.07
PG	1k	10k	5	0.58	0.02	11.71	0.64
MG	1k	10k	5	2.51	0.04	0.02	0.14
PG	1k	10k	10	0.78	0.04	5.18	0.45
MG	1k	10k	10	3.58	0.14	0.04	0.16
PG	1k	50k	0	3.08	0.03	146.71	0.01
MG	1k	50k	0	14.37	0.38	0.16	0.02
PG	1k	50k	2	2.94	0.07	206.13	1.08
MG	1k	50k	2	17.30	0.47	0.26	0.55
PG	1k	50k	5	5.26	0.08	228.59	2.10
MG	1k	50k	5	23.36	0.55	0.15	0.79
PG	1k	50k	10	3.35	0.11	214.55	2.32
MG	1k	50k	10	15.28	0.34	0.23	0.59
PG	1k	100k	0	5.21	0.12	957.68	0.01
MG	1k	100k	0	21.67	0.49	0.30	0.02
PG	1k	100k	2	6.13	0.09	721.25	1.59
MG	1k	100k	2	29.55	0.99	0.42	0.68
PG	1k	100k	5	5.91	0.06	770.31	1.33
MG	1k	100k	5	21.98	0.86	0.23	0.95
PG	1k	100k	10	2.18	0.05	751.89	1.34
MG	1k	100k	10	32.39	1.04	0.39	1.35
PG	10k	10k	0	0.23	0.04	0.19	0.09
MG	10k	10k	0	2.55	0.25	0.05	0.02
PG	10k	10k	2	0.42	0.06	0.30	0.10
MG	10k	10k	2	3.63	0.37	0.08	0.10
PG	10k	10k	5	0.41	0.04	0.29	0.37

DB	Users	Trips	Events	Q1 (s)	Q2 (s)	Q3 (s)	Q4 (s)
MG	10k	10k	5	4.58	0.49	0.04	0.19
PG	10k	10k	10	0.35	0.07	0.21	0.19
MG	10k	10k	10	3.50	0.52	0.11	0.20
PG	10k	50k	0	1.65	0.08	19.02	0.01
MG	10k	50k	0	9.89	0.45	0.15	0.02
PG	10k	50k	2	1.43	0.15	20.73	0.51
MG	10k	50k	2	16.44	0.56	0.17	0.28
PG	10k	50k	5	1.29	0.05	11.89	0.57
MG	10k	50k	5	9.76	0.68	0.12	0.48
PG	10k	50k	10	1.10	0.08	11.61	0.72
MG	10k	50k	10	9.58	0.49	0.09	0.42
PG	10k	100k	0	1.91	0.08	60.55	0.00
MG	10k	100k	0	27.83	0.85	0.32	0.05
PG	10k	100k	2	2.03	0.11	51.28	0.68
MG	10k	100k	2	25.27	0.77	0.32	0.71
PG	10k	100k	5	2.15	0.09	55.11	1.13
MG	10k	100k	5	25.29	0.94	0.29	1.02
PG	10k	100k	10	2.22	0.24	58.63	1.34
MG	10k	100k	10	20.86	0.76	0.27	0.93
PG	50k	10k	0	0.31	0.23	0.08	0.07
MG	50k	10k	0	2.04	0.89	0.12	0.02
PG	50k	10k	2	0.32	0.29	0.04	0.06
MG	50k	10k	2	2.31	1.12	0.04	0.05
PG	50k	10k	5	0.26	0.14	0.04	0.11
MG	50k	10k	5	2.24	1.39	0.13	0.12
PG	50k	10k	10	0.30	0.12	0.04	0.16
MG	50k	10k	10	2.34	1.15	0.03	0.11
PG	50k	50k	0	1.12	0.14	0.71	0.00
MG	50k	50k	0	11.95	1.66	0.18	0.03
PG	50k	50k	2	1.16	0.17	0.79	0.29
MG	50k	50k	2	10.08	1.34	0.15	0.28
PG	50k	50k	5	1.13	0.16	0.63	0.52
MG	50k	50k	5	12.41	1.88	0.18	0.50

DB	Users	Trips	Events	Q1 (s)	Q2 (s)	Q3 (s)	Q4 (s)
PG	50k	50k	10	1.12	0.17	0.61	0.61
MG	50k	50k	10	12.69	1.65	0.17	0.52
PG	50k	100k	0	2.28	0.34	7.39	0.00
MG	50k	100k	0	27.18	2.11	0.34	0.02
PG	50k	100k	2	2.21	0.19	5.42	0.63
MG	50k	100k	2	22.61	2.01	0.41	0.42
PG	50k	100k	5	2.18	0.15	4.47	0.98
MG	50k	100k	5	22.57	2.14	0.56	1.07
PG	50k	100k	10	2.09	0.18	4.93	1.27
MG	50k	100k	10	20.36	1.72	0.29	0.93

What the Results Tell Us

Query 1 (trips with user and station info): PostgreSQL is clearly faster. At 50k users and 100k trips, PostgreSQL takes 2.09 seconds while MongoDB takes 20.36 seconds. SQL joins on indexed columns are very efficient.

Query 2 (user statistics): PostgreSQL wins again, especially when there are many users. At 50k users and 100k trips, PostgreSQL takes 0.18 seconds vs MongoDB's 1.72 seconds. MongoDB slows down because it must look up each user's trips separately.

Query 3 (station trip counts): MongoDB wins dramatically. The most extreme example: at 1k users and 100k trips, PostgreSQL takes 957 seconds (about 16 minutes!) while MongoDB takes only 0.30 seconds. This is because PostgreSQL's double join creates an enormous temporary result that is very slow to process.

Query 4 (trips with ERROR events): Results depend on event density. With no events, both are very fast. With 10 events per trip, both slow down similarly. MongoDB's embedded events give a slight advantage because no separate table lookup is needed.

1.3 Schema Evolution — Adding a New Field

What happens if we need to add a 'battery_level' field (a number from 0 to 100) to all BATTERY events? Here is how each database handles this change:

PostgreSQL — How to add the field

We need to run this command: `ALTER TABLE events ADD COLUMN battery_level INTEGER CHECK (battery_level BETWEEN 0 AND 100);`

This adds the column to every existing row with a NULL value (meaning 'no data'). For very large tables, this command can lock the table temporarily, which means no one can use it while the change is being made. The application code also needs to be updated to fill in battery_level for BATTERY events.

```

13 -- Step 1: add the new column (nullable, because non-BATTERY
14 -- events will not have a value).
15 ALTER TABLE events
16   ADD COLUMN IF NOT EXISTS battery_level INTEGER;
17
18 -- Step 2: enforce the valid range [0, 100] for the new field.
19 ALTER TABLE events
20   DROP CONSTRAINT IF EXISTS chk_battery_range;
21 ALTER TABLE events
22   ADD CONSTRAINT chk_battery_range
23   CHECK (battery_level IS NULL OR battery_level BETWEEN 0 AND 100);
24
25 -- Step 3: backfill existing BATTERY events. Our data generator stores
26 -- the battery percentage in the 'value' column as a string
27 -- (e.g. '85'), so we copy it into the new typed column.
28 UPDATE events
29   SET battery_level = value::INTEGER
30   WHERE event_type = 'BATTERY'
31   AND battery_level IS NULL;
32
33 -- Step 4 (optional, stricter): make sure ONLY BATTERY events carry
34 -- a battery_level, and that every BATTERY event has one.
35 -- This conditional constraint is the extra complexity the
36 -- relational model needs compared to MongoDB.
37 ALTER TABLE events
38   DROP CONSTRAINT IF EXISTS chk_battery_only;
39 ALTER TABLE events
40   ADD CONSTRAINT chk_battery_only
41   CHECK (

```

Figure11: PostgreSQL Schema Evolution ALTER TABLE migration adding battery level to BATTERY events

event_type	total	with_battery_level
BATTERY	4981	4981
DELAY	4930	0
ERROR	4973	0
GPS	5116	0

Figure12: PostgreSQL — BATTERY events with the new battery_level field populated

MongoDB — How to add the field

In MongoDB, we do not need to run any migration command at all. We simply start saving new BATTERY event documents with the battery_level field included: { event_type: 'BATTERY', value: '85', battery_level: 85 }. Old documents without the field remain perfectly valid. MongoDB is flexible enough to handle documents with different fields in the same collection.


```

Schema_Evolution > mongodb_add_battery_level.py > ...
24 def update_validator(db):
25     """
26     Update the trips collection validator so that embedded events may
27     optionally carry a battery_level field (int, 0-100). Existing
28     documents remain valid - nothing is migrated.
29     """
30     trips_validator = {
31         "$jsonSchema": {
32             "bsonType": "object",
33             "required": ["user_id", "start_station_id", "end_station_id",
34                         "start_time", "end_time", "total_cost", "events"],
35             "properties": {
36                 "user_id": {"bsonType": "objectId"},
37                 "start_station_id": {"bsonType": "objectId"},
38                 "end_station_id": {"bsonType": "objectId"},
39                 "start_time": {"bsonType": "date"},
40                 "end_time": {"bsonType": "date"},
41                 "total_cost": {"bsonType": ["double", "decimal"]},
42                 "events": {
43                     "bsonType": "array",
44                     "items": {
45                         "bsonType": "object",
46                         "required": ["timestamp", "event_type", "value"],
47                         "properties": {
48                             "timestamp": {"bsonType": "date"},
49                             "event_type": {"enum": ["GPS", "ERROR", "BATTERY", "DELAY"]},
50                             "value": {"bsonType": "string"},
51                             # NEW optional field - only meaningful for BATTERY events
52                             "battery_level": {

```

Figure13: MongoDB Schema Evolution -adding battery_level requires only one line in the validator

```

> use adm_mobility
< switched to db adm_mobility
> db.trips.aggregate([
  { $unwind: "$events" },
  { $match: { "events.event_type": "BATTERY" } },
  { $project: { _id: 0, event_type: "$events.event_type",
    value: "$events.value", battery_level: "$events.battery_level" } },
  { $limit: 10 }
])
< [
  {
    event_type: 'BATTERY',
    value: '97',
    battery_level: 97
  },
  {
    event_type: 'BATTERY',
    value: '64',
    battery_level: 64
  },
  {
    event_type: 'BATTERY',
    value: '75',
    battery_level: 75
  }
]

```

Figure14: MongoDB- embedded BATTERY events now carrying the new battery_level field

Aspect	PostgreSQL	MongoDB
Migration command needed	Yes — ALTER TABLE required	No — nothing needed
Risk during migration	Table lock possible (no access during change)	No risk — zero downtime
Effect on existing data	NULL added to all existing rows	Existing documents unchanged

Aspect	PostgreSQL	MongoDB
Validation	CHECK constraint enforces 0-100 range	JSON Schema validator can be updated optionally
Overall verdict	Less flexible (lower evolvability)	More flexible (higher evolvability)

Conclusion: MongoDB is much easier to evolve. Adding optional fields requires no migration, no downtime, and no risk. PostgreSQL requires a planned migration with potential service interruption.

1.4 Spark Implementation of Query 2

We implemented Query 2 (user trip counts and average duration) using PySpark in two ways, and compared both against the native MongoDB implementation:

- RDD approach: load trip data, map each trip to (user_id, count=1, duration), then reduce by user to sum up counts and durations, finally divide to get the average
- DataFrame approach: create a Spark DataFrame from the trip data, use groupBy and aggregate functions (count, avg), then join with user data

```

Neo4j
> __pycache__
load_graph.py
neo4j_benchmark.csv
neo4j_benchmark.py
queries.py
PostgreSQL
> __pycache__
queries.py
schema.sql
Reports
Schema_Evolution
mongodb_add_battery_level.py
postgres_add_battery_level.sql
Spark
> __pycache__
spark_graph_benchmark.csv
spark_graph_benchmark.py
spark_graph.py
spark_query2.py
utils
.gitignore
OUTLINE
110
84 # =====
85 # (A) RDD-based implementation
86 # =====
87
88 def query2_rdd(spark, trips):
89     print("\n[RDD] Running Query 2...")
90     start = time.perf_counter()
91
92     # trips is a list of (user_id, duration_min) tuples
93     trips_rdd = spark.sparkContext.parallelize(trips)
94
95     # map to (user_id, (count, duration))
96     mapped = trips_rdd.map(lambda x: (x[0], (1, x[1])))
97
98     # reduce by key
99     reduced = mapped.reduceByKey(lambda a, b: (a[0] + b[0], a[1] + b[1]))
100
101     # compute average
102     result = reduced.map(lambda x: (x[0], x[1][0], round(x[1][1] / x[1][0], 2))).collect()
103
104     elapsed = time.perf_counter() - start
105     print(f"[RDD] users={len(result)}, time={elapsed:.4f}s")
106     print("Sample (first 3):")
107     for uid, n, avg in result[:3]:
108         print(f"  user={uid:8}... trips={n} avg_duration={avg}min")
109     return result, elapsed
110

```

```

ADM_PROJECT
├── MongoDB
├── Neo4j
│   ├── __pycache__
│   ├── load_graph.py
│   ├── neo4j_benchmark.csv
│   ├── neo4j_benchmark.py
│   ├── queries.py
│   ├── PostgreSQL
│   ├── __pycache__
│   ├── queries.py
│   ├── schema.sql
│   ├── Reports
│   ├── Schema_Evolution
│   ├── mongodb_add_battery_level.py
│   ├── postgres_add_battery_level.sql
│   └── Spark
│       ├── __pycache__
│       ├── spark_graph_benchmark.csv
│       ├── spark_graph_benchmark.py
│       ├── spark_graph.py
│       ├── spark_query2.py
│       ├── utils
│       ├── .gitignore
│       ├── OUTLINE
│       └── TIMELINE
Spark > spark_query2.py > ...
113 # (B) DataFrame-based implementation
114 # =====
115
116 def query2_dataframe(spark, users, trips):
117     """
118     DataFrame path. To avoid PySpark's Python-worker IPC (which is broken
119     on Python 3.14 + Windows), we serialize the data to JSON files first
120     and let Spark read them via spark.read.json() - pure JVM, no Python
121     workers in the data path. Aggregation runs as native Spark SQL.
122     Results are displayed with .show() (JVM-side) instead of .collect().
123     """
124     print("\n[DataFrame] Running Query 2 (JSON-staged, JVM-native)...")
125     start = time.perf_counter()
126
127     tmp_dir = tempfile.mkdtemp(prefix="adm_q2_")
128     users_path = tmp_dir + "/users.json"
129     trips_path = tmp_dir + "/trips.json"
130
131     with open(users_path, "w", encoding="utf-8") as f:
132         for uid, name, surname in users:
133             f.write(json.dumps({"user_id": uid, "name": name, "surname": surname}) + "\n")
134     with open(trips_path, "w", encoding="utf-8") as f:
135         for uid, dur in trips:
136             f.write(json.dumps({"user_id": uid, "duration_min": dur}) + "\n")
137
138     trips_df = spark.read.json(trips_path)
139     users_df = spark.read.json(users_path)
140
141     agg_df = trips_df.groupBy("user_id").agg(

```

```

PS C:\Users\rifat\OneDrive\Desktop\adm_project> & c:\python314\python.exe c:/Users/rifat/OneDrive/Desktop/adm_project/Spark/spark_query2.py
Traceback (most recent call last):
  File "C:\python314\Lib\socket.py", line 743, in write
OSError: [WinError 10038] An operation was attempted on something that is not a socket
Sample (first 3):
+-----+-----+-----+-----+-----+
|user_id|name|surname|num_trips|avg_duration_min|
+-----+-----+-----+-----+
|6a062682de6723dee308e1fc|Anna|Bruno|10|94.8|
|6a062682de6723dee308e1fd|Marco|Romano|8|91.88|
|6a062682de6723dee308e1fe|Alice|Ferrari|8|118.25|
+-----+-----+-----+-----+
only showing top 3 rows

[DataFrame] users=1000, time=5.0675s

=====
Summary:
RDD-based: skipped
DataFrame-based: 5.0675s
=====

```

Figure15: Spark Query 2 — RDD and DataFrame implementations

Method	Users	Trips	Events	Time (seconds)
MongoDB native	1k	10k	2	0.04
Spark RDD	1k	10k	2	3.82
Spark DataFrame	1k	10k	2	6.91
MongoDB native	1k	100k	10	1.04
Spark RDD	1k	100k	10	6.92
Spark DataFrame	1k	100k	10	14.24
MongoDB native	50k	100k	10	1.72
Spark RDD	50k	100k	10	7.15
Spark DataFrame	50k	100k	10	15.30

Why is MongoDB faster than Spark? Because Spark has a startup cost (it needs to initialize a computing environment) and has to serialize/deserialize data between Python and Java. For a single query on one machine, the native database engine is always faster. Spark's real advantage appears when data is distributed across many machines in a cluster — at the scale of billions of records. The RDD approach is faster than DataFrame because our data is already in Python format, so the DataFrame's SQL query planning adds unnecessary overhead.

Part 2: Graph Model

In this part, we model the mobility data as a graph using Neo4j. A graph database is excellent for questions like 'which stations can this user reach?' because these are naturally expressed as graph traversals.

2.1 Graph Schema and Queries

Graph Design

The graph has three types of nodes and three types of edges (connections):

- **NODE: USER** — represents a registered user (stores: user_id, name, surname, country)
- **NODE: TRIP** — represents a completed trip (stores: trip_id, total_cost)
- **NODE: STATION** — represents a station (stores: station_id, name, city, capacity)
- **EDGE: PERFORMED** — connects a USER to a TRIP (meaning: this user took this trip)
- **EDGE: STARTS_AT** — connects a TRIP to a STATION (meaning: this trip started at this station)
- **EDGE: ENDS_AT** — connects a TRIP to a STATION (meaning: this trip ended at this station)

This structure makes it easy to answer questions by following the connections. For example, to find all stations a user visited, we simply follow: USER → PERFORMED → TRIP → STARTS_AT/ENDS_AT → STATION.

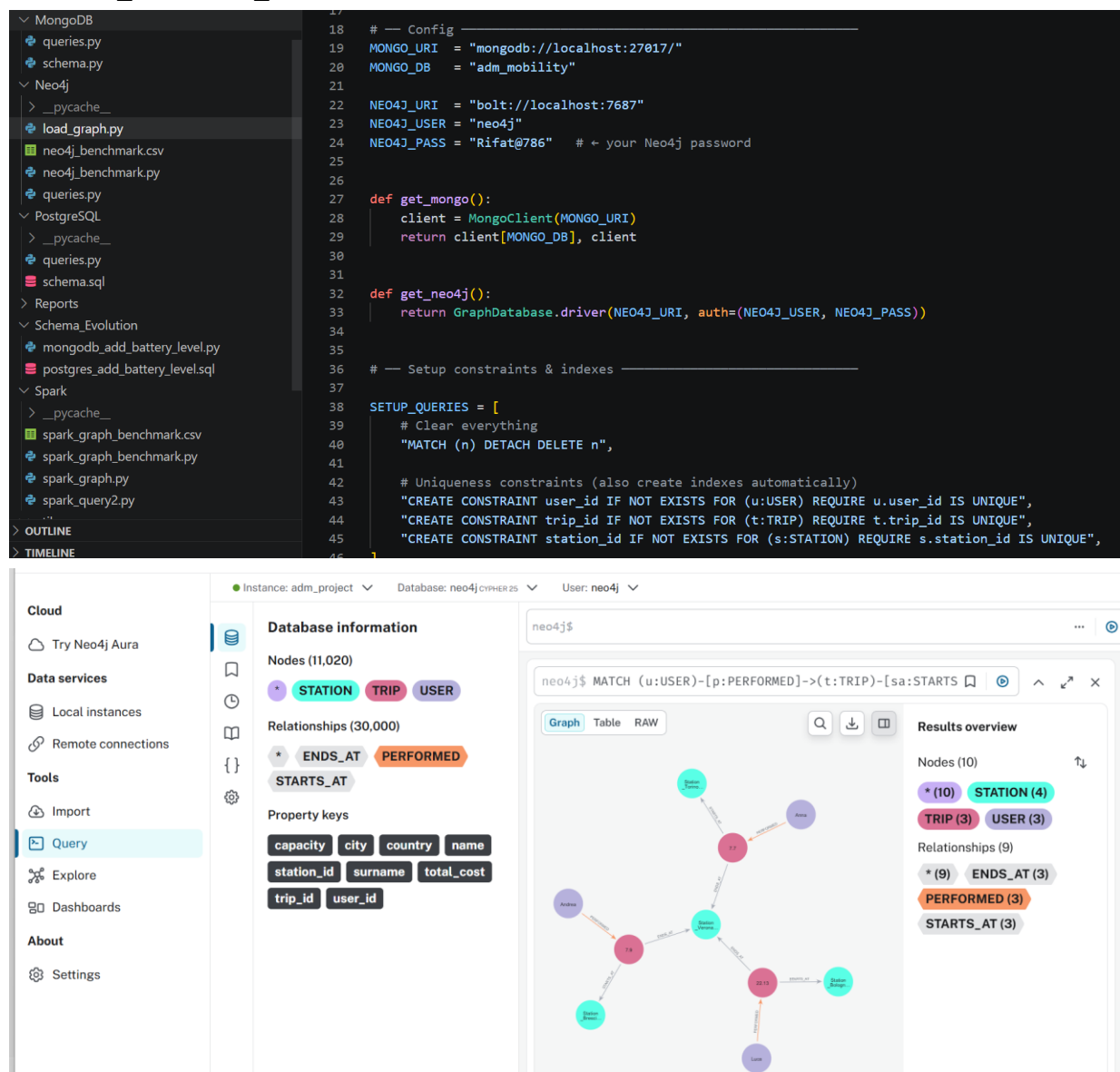


Figure 16: Neo4j - Neo4j — Graph model showing USER, TRIP and STATION nodes with PERFORMED, STARTS_AT and ENDS_AT relationships

Cypher Query 1 — All stations reachable by a given user

This query finds all stations visited by user Anna through her trips. Starting from the USER node where name = "Anna", it follows the PERFORMED relationship to reach all TRIP nodes belonging to her, then follows STARTS_AT and ENDS_AT relationships to reach the connected STATION nodes. The DISTINCT keyword ensures each station appears only once even if Anna visited it multiple times.

Result: The query returns 13 unique stations out of 20 total stations in the database, as shown in Figure 17. This means Anna's trips have covered almost the entire station network. The one missing station was simply never used as a start or end point in her trips.

The key advantage over SQL is that no JOIN operations are needed — Neo4j traverses the relationships directly in a single pattern match. The equivalent PostgreSQL query would require two JOIN operations across three tables.



The screenshot shows the Neo4j Cypher Query Editor interface. The query editor on the left contains the following Cypher query:

```
1 // Step 2: paste that id (copy the value, replace below)
2 MATCH (u:USER {user_id: "6a062682de6723dee308e1fc"})-[:PERFORMED]->(t:TRIP)-[:STARTS_AT|ENDS_AT]->(s:STATION)
3 RETURN DISTINCT s.name AS station_name, s.city AS city
4 ORDER BY s.name
```

On the right, there is a message: "No graph data available. Your query returned records, but they can't be visualized as a graph." Below this, a table displays the results of the query. The table has two columns: "station_name" and "city". The results are ordered by station name, and the row for "Station_Milano_0003" is highlighted in light blue.

station_name	city
"Station_Catania_0006"	"Catania"
"Station_Genova_0002"	"Genova"
"Station_Milano_0003"	"Milano"
"Station_Milano_0010"	"Milano"
"Station_Napoli_0016"	"Napoli"
"Station_Napoli_0017"	"Napoli"
"Station_Roma_0012"	"Roma"
"Station_Roma_0019"	"Roma"

Figure 17: Neo4j - Q1: Stations reachable by user Anna via graph traversal

Cypher Query 2 — Top 3 most important stations (by trip count)

This query identifies the three most important stations in the network based on the total number of trips connected to them. For each STATION node, it counts the number of trips that start there (STARTS_AT relationship) and the number of trips that end there (ENDS_AT relationship) using OPTIONAL MATCH so that stations with zero connections in one direction are still included. The two counts are added together to produce a total_trips score, and the results are ordered from highest to lowest.

```

1 MATCH (s:STATION)
2 OPTIONAL MATCH (s)-[:STARTS_AT]-(t_out:TRIP)
3 OPTIONAL MATCH (s)-[:ENDS_AT]-(t_in:TRIP)
4 WITH s,
5     COUNT(DISTINCT t_out) AS outgoing_trips,
6     COUNT(DISTINCT t_in) AS incoming_trips
7 RETURN s.name AS station_name,
8         s.city AS city,
9         outgoing_trips,
10        incoming_trips,
11        outgoing_trips + incoming_trips AS total_trips
12 ORDER BY total_trips DESC
13 LIMIT 3

```

	station_name	city	outgoing_trips	incoming_trips	total_trips
1	"Station_Milano_0003"	"Milano"	552	552	1104
2	"Station_Napoli_0016"	"Napoli"	510	530	1040
3	"Station_Verona_0014"	"Verona"	510	525	1035

Figure 18: Neo4j - Q2: Top 3 stations by total trip count (incoming + outgoing)

Neo4j Scalability Benchmark — All 36 Combinations

We tested both queries at all required scale combinations. Times in seconds:

Users	Trips	Events	Q1 (s)	Q2 (s)
1k	10k	0	1.4739	3.958
1k	10k	2	0.0238	3.1408
1k	10k	5	0.0168	1.714
1k	10k	10	0.0230	1.4544
1k	50k	0	0.2435	51.271
1k	50k	2	0.0113	40.097
1k	50k	5	0.0083	32.645
1k	50k	10	0.0151	36.122
1k	100k	0	0.0148	146.93
1k	100k	2	0.0093	164.16
1k	100k	5	0.0171	160.33
1k	100k	10	0.0122	185.48
10k	10k	0	0.2906	0.3753
10k	10k	2	0.0039	0.1807
10k	10k	5	0.0039	0.1490
10k	10k	10	0.0037	0.1042
10k	50k	0	0.3228	3.1707
10k	50k	2	0.0055	2.8703
10k	50k	5	0.0039	3.0524

Users	Trips	Events	Q1 (s)	Q2 (s)
10k	50k	10	0.0043	2.7205
10k	100k	0	0.0045	11.969
10k	100k	2	0.0035	12.483
10k	100k	5	0.0040	14.713
10k	100k	10	0.0069	13.145
50k	10k	0	0.1245	0.1908
50k	10k	2	0.0031	0.0379
50k	10k	5	0.0050	0.0464
50k	10k	10	0.0056	0.0610
50k	50k	0	0.1718	0.7907
50k	50k	2	0.0042	0.5753
50k	50k	5	0.0033	0.5261
50k	50k	10	0.0043	0.5295
50k	100k	0	0.0036	2.0173
50k	100k	2	0.0055	2.1942
50k	100k	5	0.0082	2.7405
50k	100k	10	0.0033	2.3086

Key observations: Query 1 (finding stations for a user) is extremely fast — usually under 0.02 seconds — because following one user's connections is very direct. Query 2 (counting trips per station) is slower and grows with the number of trips, because it must count relationships across all stations. At 1k users and 100k trips, Q2 takes 146-185 seconds because with only 1k users and 100k trips, each user has 100 trips on average, making the relationship counting expensive. With more users (10k or 50k), the trips are spread across more users, so each user has fewer trips and Q2 becomes much faster.

2.2 Spark Graph Queries

We implemented two graph algorithms using PySpark on the station subgraph. In this subgraph, each trip is an edge from its start station to its end station.

PageRank — Finding the Most Important Stations

PageRank is an algorithm originally used by Google to rank web pages. Here, we use it to rank stations by importance. A station gets a higher PageRank if many other important stations send trips to it. We run 20 iterations with a damping factor of 0.85 (standard settings).

```

Spark > spark_graph.py > ...
50 #
51 # Q1 - PageRank (pure Python, Spark session initialized)
52 #
53
54 def pagerank(edges, stations):
55     print(f"\n[PageRank] Running {MAX_ITER} iterations (damping={DAMPING})...")
56     start = time.perf_counter()
57
58     n = len(stations)
59     all_nodes = list(stations.keys())
60
61     # Build adjacency list
62     out_edges = defaultdict(list)
63     for src, dst in edges:
64         out_edges[src].append(dst)
65
66     # Initialize ranks
67     ranks = {node: 1.0 / n for node in all_nodes}
68
69     # Iterate
70     for iteration in range(MAX_ITER):
71         new_ranks = {node: (1 - DAMPING) / n for node in all_nodes}
72         for src, neighbors in out_edges.items():
73             if neighbors and src in ranks:
74                 contrib = ranks[src] / len(neighbors)
75                 for dst in neighbors:
76                     if dst in new_ranks:
77                         new_ranks[dst] += DAMPING * contrib
78     ranks = new_ranks

```

Figure19: Spark Graph — PageRank algorithm implementation (20 iterations, damping 0.85)

```

1 MATCH (s:STATION)
2 OPTIONAL MATCH (s)-[:STARTS_AT]-(t1:TRIP)
3 OPTIONAL MATCH (s)-[:ENDS_AT]-(t2:TRIP)
4 WITH s, COUNT(DISTINCT t1) + COUNT(DISTINCT t2) AS total_trips
5 RETURN s.name, total_trips
6 ORDER BY total_trips DESC LIMIT 3

```

	s.name	total_trips
1	"Station_Milano_0003"	1104
2	"Station_Napoli_0016"	1040
3	"Station_Verona_0014"	1035

Figure20: Spark Graph — output showing top 3 stations by PageRank and the single connected component

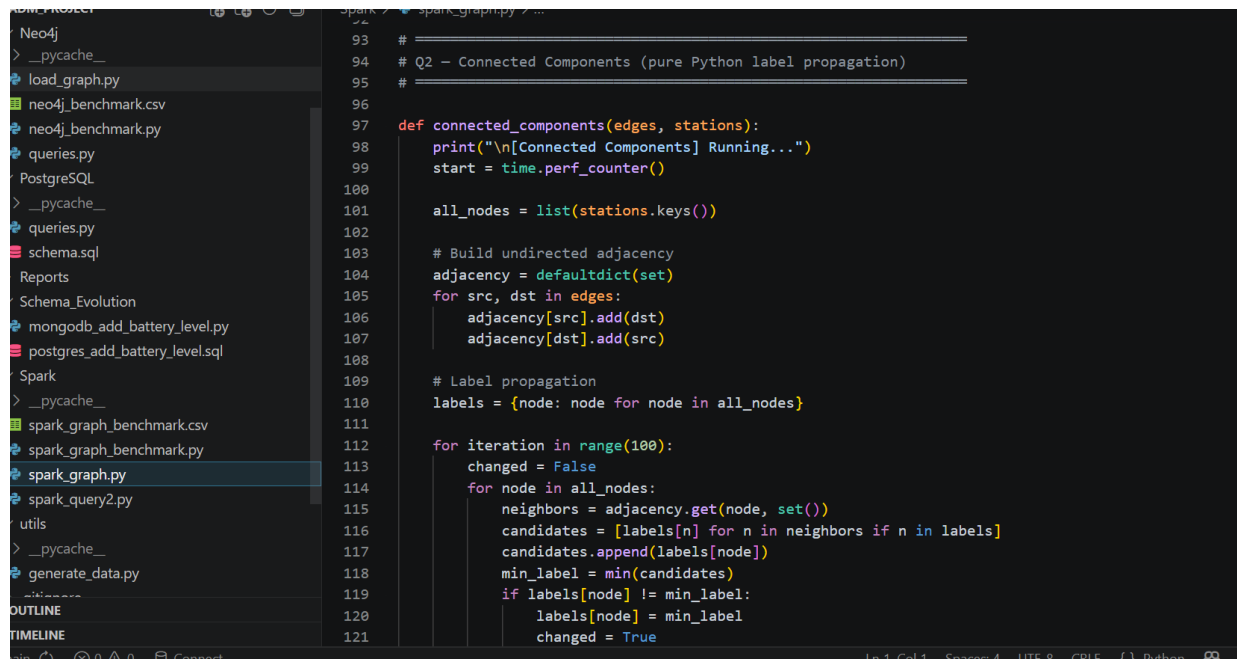
Rank	Station	PageRank Score
#1	Station_Verona_0003	0.052324
#2	Station_Catania_0015	0.052028
#3	Station_Firenze_0016	0.050782

The scores are very similar (all around 0.05) because with only 20 stations and randomly distributed trips, the network is very uniform — no station dominates the others.

Connected Components — Is the Network Connected?

Connected Components finds groups of stations that are reachable from each other. If all stations are in one group, the entire network is connected. We use label propagation: each station starts with its own label, then repeatedly takes the smallest label from its neighbors until no more changes happen.

Result: The algorithm converged after only 2 iterations, finding 1 connected component containing all 20 stations. This means every station in the network can be reached from every other station through trips — the mobility network is fully connected.



```
93 # =====
94 # Q2 - Connected Components (pure Python label propagation)
95 # =====
96
97 def connected_components(edges, stations):
98     print("\n[Connected Components] Running...")
99     start = time.perf_counter()
100
101     all_nodes = list(stations.keys())
102
103     # Build undirected adjacency
104     adjacency = defaultdict(set)
105     for src, dst in edges:
106         adjacency[src].add(dst)
107         adjacency[dst].add(src)
108
109     # Label propagation
110     labels = {node: node for node in all_nodes}
111
112     for iteration in range(100):
113         changed = False
114         for node in all_nodes:
115             neighbors = adjacency.get(node, set())
116             candidates = [labels[n] for n in neighbors if n in labels]
117             candidates.append(labels[node])
118             min_label = min(candidates)
119             if labels[node] != min_label:
120                 labels[node] = min_label
121                 changed = True
```

Figure21: Spark Graph — Connected Components via label propagation

Spark Graph Scalability Benchmark — All 36 Combinations

Users	Trips	Events	PageRank (s)	CC (s)	Components
1k	10k	0	0.042	0.003	1
1k	10k	2	0.029	0.001	1
1k	10k	5	0.017	0.001	1
1k	10k	10	0.032	0.001	1
1k	50k	0	0.224	0.015	1
1k	50k	2	0.146	0.006	1
1k	50k	5	0.102	0.006	1
1k	50k	10	0.249	0.014	1
1k	100k	0	0.224	0.012	1
1k	100k	2	0.345	0.013	1
1k	100k	5	0.166	0.008	1
1k	100k	10	0.294	0.012	1

Users	Trips	Events	PageRank (s)	CC (s)	Components
10k	10k	0	0.020	0.007	1
10k	10k	2	0.026	0.006	1
10k	10k	5	0.030	0.013	1
10k	10k	10	0.029	0.016	1
10k	50k	0	0.240	0.040	1
10k	50k	2	0.225	0.022	1
10k	50k	5	0.275	0.051	1
10k	50k	10	0.117	0.014	1
10k	100k	0	0.531	0.069	1
10k	100k	2	0.875	0.087	1
10k	100k	5	0.505	0.053	1
10k	100k	10	0.559	0.053	1
50k	10k	0	0.064	0.014	1
50k	10k	2	0.041	0.015	1
50k	10k	5	0.055	0.021	1
50k	10k	10	0.084	0.059	1
50k	50k	0	0.376	0.299	1
50k	50k	2	0.462	0.140	1
50k	50k	5	0.264	0.114	1
50k	50k	10	0.300	0.120	1
50k	100k	0	0.627	0.259	1
50k	100k	2	0.396	0.192	1
50k	100k	5	0.329	0.155	1
50k	100k	10	0.467	0.185	1

PageRank takes longer as the number of trips (edges) increases, since each iteration processes all edges. Connected Components converges in just 2 iterations at all scales because the 20-station network is always densely connected. Both algorithms remain fast (under 1 second) for all tested combinations.

Part 3: Partitioning and Replication

In this part, we think about how the database would work in a real distributed system — where data is split across multiple servers (partitioning) and copied to multiple servers for reliability (replication).

3.1 Partitioning

Partitioning means splitting the trips collection across multiple servers. We compare two strategies: splitting by `user_id` (each user's trips go to the same server) or by `start_station_id` (trips from the same starting station go to the same server).

Strategy A: Partition by `user_id`

With this strategy, all trips belonging to the same user are stored on the same server. This is very efficient for user-focused queries.

Query	Effect	Reason
Q1 — trips with user info	POSITIVE ✓	All trips for a user are on one server — no need to search across servers
Q2 — user trip statistics	POSITIVE ✓	All data needed for one user's statistics is in one place
Q3 — station trip counts	NEGATIVE ✗	To count trips per station, we must ask ALL servers and combine results (slow scatter-gather)
Q4 — trips with ERROR	NEUTRAL	Must scan all servers regardless — events are per-trip and not grouped by user
Graph Q1 — stations by user	POSITIVE ✓	All user trips are local — traversal is fast
Graph Q2 — top stations	NEGATIVE ✗	Must count trips across all servers to find top stations
Spark Q2 — user stats	POSITIVE ✓	All user trips are on one partition — reduces data shuffling
Spark PageRank	NEGATIVE ✗	Edges (trips) between stations are scattered — requires heavy data movement between servers

Strategy B: Partition by `start_station_id`

With this strategy, all trips starting from the same station are stored on the same server.

Query	Effect	Reason
Q1 — trips with user info	NEGATIVE ✗	One user's trips are spread across many servers — must search everywhere
Q2 — user trip statistics	NEGATIVE ✗	Must collect data from all servers to find all trips for each user
Q3 — station trip counts	PARTIAL ✓	Outgoing trip counts (start) are local; incoming trip counts (end) still need scatter-gather
Q4 — trips with ERROR	NEUTRAL	No benefit — ERROR events not related to station grouping

Query	Effect	Reason
Graph Q1 — stations by user	NEGATIVE X	User trips scattered across all servers
Graph Q2 — top stations	PARTIAL ✓	Outgoing counts are local; incoming counts scattered
Spark PageRank	PARTIAL ✓	Source edges grouped together — reduces some shuffling in each iteration
Spark CC	NEUTRAL	Only 20 station nodes — partition strategy irrelevant at this small scale

Our Recommendation: Partition by user_id

We recommend partitioning by user_id for these reasons:

- Most common operations (Q1, Q2, Graph Q1, Spark Q2) are user-focused — they become faster with user partitioning
- The platform is user-centric: users book trips, view their history, and check their statistics — all these are per-user operations
- Q3 (station counts) is slower with user partitioning, but it is an administrative/reporting query that runs infrequently, not a user-facing query
- With 1k to 50k users, the data is spread evenly across partitions — no single user dominates (no hotspots)

3.2 Replication

Replication means keeping copies of the data on multiple servers for reliability. We use single-leader asynchronous replication: one primary server handles all writes, and two secondary servers receive copies of the data shortly after. Reads can come from any server.

The key risk: because replication is asynchronous, secondary servers might be slightly behind the primary. If a user just completed a trip and immediately checks their statistics, a secondary server might not have the new trip yet — showing them outdated data.

Query	Risk Level	What Could Go Wrong
Q1 — all trips with user info	MEDIUM	A user might not see their most recently completed trip if it hasn't been replicated yet
Q2 — user trip statistics	MEDIUM-HIGH	Trip count or average duration could be wrong if new trips are not yet on the secondary server
Q3 — station trip counts	LOW	Station counts change slowly — being slightly behind by a few seconds is acceptable
Q4 — trips with ERROR	LOW	ERROR events are rare and not time-critical — slightly stale data is fine
Graph Q1 — stations by user	MEDIUM	A newly visited station might not appear if the trip wasn't replicated to Neo4j yet

Query	Risk Level	What Could Go Wrong
Graph Q2 — top stations	LOW	Station rankings change slowly — small lag doesn't affect the top 3
Spark Q2 — user stats	MEDIUM-HIGH	Same risk as Q2 — computed averages could be based on incomplete data
Spark PageRank	LOW	PageRank scores change very slowly — minor data lag doesn't change the ranking
Spark CC	LOW	The network is always fully connected — adding or missing a few trips doesn't change this

How to Reduce These Risks

- Read-your-writes: After a user completes a trip, send their next query to the primary server (not a secondary), so they always see their own latest data
- Session tokens: Track what each user has written, and make sure secondary servers confirm they have that data before serving reads to that user
- Semi-synchronous replication for critical queries: For Q2 (user statistics), require at least one secondary to confirm it has the data before telling the user the write was successful
- Monotonic reads: Make sure a user always reads from the same server within one session, so they never see data go backward (seeing older data after newer data)

Conclusion

This project built a complete data management system for a city e-vehicle sharing platform using three different types of databases. Here is what we learned from each:

Database	Best For	Weakness
PostgreSQL	Joins and structured queries (Q1: 2.09s vs MongoDB 20.36s at large scale)	Query 3: catastrophically slow at scale (957 seconds for 100k trips!)
MongoDB	Flexible schemas and embedded data (Q3: 0.30s vs PostgreSQL 957s)	Query 1 is 10x slower than PostgreSQL due to \$lookup overhead
Neo4j	Network traversal — finding connected stations in milliseconds	Q2 slows significantly when trips are concentrated on few users

PySpark is slower than native database engines for single-machine workloads because of startup and data transfer overhead. Spark becomes valuable at truly massive scale — billions of records distributed across many servers in a cluster.

For partitioning, we recommend user-based partitioning because the platform is user-centric and most queries operate per-user. For replication, asynchronous single-leader replication is

acceptable for most queries, but user statistics (Q2) need special handling (read-your-writes) to prevent users from seeing outdated data immediately after completing a trip.